Practical Lab: Comparing Telnet and SSH Traffic in Wireshark

Skill level: Beginner **Focus:** Network security, packet capture, encryption

Lab goal
Capture two local remote-access sessions with Wireshark and compare what can be observed when a session uses Telnet versus SSH. The main security lesson is that Telnet exposes application data in plaintext, while SSH protects the session with encryption.

Learning objectives

By the end of this lab, you should be able to:

- Start a Wireshark packet capture on a loopback interface.
- Generate a Telnet session and identify its packets.
- Follow a Telnet TCP stream and recognize visible application data.
- Generate an SSH session and identify SSH traffic.
- Follow an SSH TCP stream and recognize that the session contents are not readable as plaintext.
- Explain why SSH is preferred for secure remote administration.

Required environment

- A Linux virtual machine or lab workstation with Wireshark installed.
- A working local Telnet service for the Telnet portion.
- A working local SSH service for the SSH portion.
- A terminal with permission to start Wireshark and connect to localhost.

Important lab note
Use accounts and passwords created specifically for your lab. Do not place real credentials or production secrets into packet captures or screenshots.

Part 1 — Observe a Telnet Session

Telnet is used here only as a controlled demonstration of why plaintext remote administration is insecure.

Step 1: Start Wireshark and capture traffic

1. Open a terminal and launch Wireshark.

```
wireshark &
```

2. Select the loopback interface (often named **lo**) and begin capturing.

3. Open a second terminal and connect to the local Telnet service.

```
telnet localhost
```

4. Sign in with your dedicated lab account when prompted. Type a harmless test command such as:

```
whoami
```

5. Stop the capture after completing the test login and command.

Step 2: Filter and inspect Telnet traffic

1. In the Wireshark display filter box, enter:

```
telnet
```

2. Select a Telnet packet, right-click it, and choose **Follow** → **TCP Stream**.

3. Examine the stream carefully. In a Telnet session, application data can be visible as readable text. This is the key security observation of this part of the lab.

4. Close the stream view and end the Telnet session.

```
exit
```

Part 2 — Observe an SSH Session

SSH provides secure remote access by encrypting the session. We will repeat a similar capture and compare the results.

Step 1: Start a new Wireshark capture

1. Start a fresh capture on the loopback interface.
2. In another terminal, connect to the local SSH service.

```
ssh localhost
```

3. If the host-key confirmation appears, review it and accept it only for this controlled lab environment if appropriate.
4. Authenticate with your dedicated lab account.
5. Run a harmless command, for example:

```
whoami
```

6. Stop the Wireshark capture.

Step 2: Filter and inspect SSH traffic

1. Apply this Wireshark display filter:

```
ssh
```

2. Locate an SSH packet and choose **Follow** → **TCP Stream**.
3. Compare the stream with the Telnet stream you examined earlier. The SSH session should not expose the application data as readable plaintext because the session is encrypted.

Part 3 — Compare the Evidence

Observation	Telnet	SSH
Traffic identification	Use the Telnet display filter	Use the SSH display filter
Can application data be read directly?	Typically visible as plaintext	Protected by encryption
Credential exposure risk	High on an untrusted network	Designed to protect credentials and session data
Recommended for secure administration?	No	Yes

Part 4 — Reflection Questions

1. What information was visible when you followed the Telnet TCP stream?
2. What changed when you followed the SSH TCP stream?
3. Why is sending authentication information over an unencrypted session dangerous?
4. What security property does SSH provide that Telnet does not?
5. Why should packet captures containing credentials never be uploaded to a public GitHub repository?

Portfolio documentation checklist

For a GitHub project, document the lab using your own screenshots and your own wording. Include the environment used, commands executed, Wireshark filters, observations, and a short comparison of Telnet and SSH. Redact usernames, passwords, tokens, private IP addresses, hostnames, and other information that should not be public.

Suggested project structure

```
telnet-vs-ssh-wireshark-lab/
■■■ README.md
■■■ commands_used.txt
■■■ screenshots/
■ ■■■ telnet-filter.png
■ ■■■ telnet-stream.png
■ ■■■ ssh-filter.png
■ ■■■ ssh-stream.png
■■■ notes/
■■■ observations.md
```

Safety and ethics
Perform the captures only on systems and accounts you are authorized to use. The purpose of this exercise is to understand network security and the protection provided by encryption.